# Software-Engineering-Portfolio

## Lern- und Projektplan für eine professionelle Full-Stack-Anwendung

Ausgangspunkt: Java/OOP vorhanden, SQL und relationale Datenbanken vorhanden, grundlegende Linux-/Terminal-Kenntnisse und erste Kenntnisse in Netzwerken/Protokollen. Ziel ist nicht nur eine funktionierende App, sondern ein Projekt, an dem Architektur, Datenbanken, Web, Netzwerke, Security, Testing, Deployment und Software Engineering praktisch gelernt werden.

Arbeitsprinzip: Du implementierst selbst. Ich unterstütze als Wegweiser, Reviewer und Debugging-Partner. Bei Architektur- und Designfragen sollst du zuerst eine eigene Lösung skizzieren; danach prüfen wir sie gemeinsam.

# 1. Projektidee

Arbeitstitel: AthleteOS – eine Plattform für Training, persönliche Ziele, Aufgaben, Ernährung/Statistiken, Bilder und Kommunikation.

Die Anwendung wird absichtlich so entwickelt, dass sie später mehrere technische Ebenen enthält: Web-Frontend, Java-Backend, relationale Datenbank, Authentifizierung, REST, WebSockets, Dateispeicher, Redis, Hintergrundjobs, Tests, Docker, Linux und CI/CD.

Wichtig: Nicht alles wird am Anfang gebaut. Das Projekt wächst in Versionen. Zuerst entsteht eine kleine funktionierende Anwendung, danach wird sie schrittweise professioneller.

# 2. Benutzerrollen

| Rolle | Zweck |
|---|---|
| Normaler Benutzer | Trainiert, verwaltet eigenen Trainingsplan, Trainingsdaten, Aufgaben, Bilder und Profil; kann mit anderen Benutzern interagieren. |
| Trainer | Kann – abhängig von den später festgelegten Berechtigungen – Trainingspläne für zugewiesene Benutzer erstellen/ändern und deren Fortschritt einsehen. |
| Admin | Verwaltet Benutzer, Rollen und Systemfunktionen; besitzt administrative Rechte. |
| Developer/Test Account | Nur für Entwicklung und automatisierte Tests. Nicht als normale Produktionsrolle behandeln; idealerweise getrennte Testdaten/Testumgebung. |

# 3. Welche Daten brauchen wir?

Deine Liste ist ein guter Anfang. Einige Daten sollten wir ergänzen, weil sie für die Funktionalität oder für saubere Datenmodelle notwendig werden.

| Bereich | Mögliche Daten | Warum? |
|---|---|---|
| User | id, Vorname, Nachname, E-Mail, Passwort-Hash, Geburtsdatum/Alter, Adresse, created_at, updated_at | Identität, Anmeldung, Profil und Nachvollziehbarkeit. Das Passwort selbst wird niemals im Klartext gespeichert. |
| Rolle | role / permissions | Damit der Server zwischen Benutzer, Trainer und Admin unterscheiden kann. |
| Training | Workout, Datum, Dauer, Status | Ein Benutzer soll Trainingseinheiten speichern und später wieder abrufen können. |
| Übung | Exercise, Name, Muskelgruppe, Beschreibung | Übungen sind wiederverwendbare Daten und sollten nicht in jedem Workout als Freitext dupliziert werden. |

| Bereich | Mögliche Daten | Warum? |
|---|---|---|
| Satz | Gewicht, Wiederholungen, Satznummer | Konkrete Trainingsleistung; typischerweise gehört ein Satz zu einer Übung innerhalb eines Workouts. |
| Trainingsplan | Name, Beschreibung, Ziel, Übungen | Trainer und Benutzer können strukturierte Pläne verwalten. |
| Zuweisung | Trainer ↔ Benutzer | Ein Trainer kann mehrere Benutzer betreuen; je nach Regel kann ein Benutzer mehrere Trainer haben. Das führt ggf. zu M:N. |
| Körpergewicht | Gewicht, Datum | Zeitreihe für Fortschrittsauswertung. |
| Aufgabe | Titel, Beschreibung, Status, Priorität, Fälligkeitsdatum | Task-Management; kann später einem Benutzer zugewiesen werden. |
| Nachricht | Sender, Empfänger, Text, created_at, edited_at, read_at | Private Kommunikation und Status der Nachricht. |
| Bild | Owner, Speicherort/Storage-Key, MIME-Type, Größe, created_at | Bilder werden besser im Object Storage abgelegt; die DB speichert Metadaten. |
| Ziel | Typ, Zielwert, Startdatum, Enddatum, Status | Persönliche Ziele und Fortschrittsmessung. |
| Benachrichtigung | Empfänger, Typ, Text/Referenz, gelesen | Später nützlich für WebSockets und asynchrone Events. |

# 4. Was mit „Beziehungen“ gemeint ist

Du hast richtig vermutet, dass wir über 1:n und M:n sprechen. Aber Beziehungen gibt es nicht nur zwischen Benutzertypen. Gemeint sind die Beziehungen zwischen den Entitäten/Datenobjekten der Anwendung.

Beispiele:

• Ein Benutzer → viele Workouts: 1:n. Ein Workout gehört einem Benutzer; ein Benutzer kann viele Workouts haben.

• Ein Workout → viele Sätze: 1:n. Ein Workout enthält mehrere gespeicherte Sätze.

• Ein Trainer ↔ viele Benutzer: möglicherweise M:n. Ein Trainer betreut viele Benutzer und ein Benutzer könnte von mehreren Trainern betreut werden. Dafür wäre eine Zwischentabelle wie trainer_user nötig.

• Workout ↔ Exercise: häufig M:n. Eine Übung kann in vielen Workouts vorkommen und ein Workout enthält viele Übungen. Eine Zwischentabelle kann zusätzliche Daten wie Reihenfolge oder Wiederholungsziel speichern.

• Benutzer → viele Nachrichten: 1:n aus Sicht des Senders und 1:n aus Sicht des Empfängers. Deshalb braucht die Nachricht zwei Fremdschlüssel auf User.

Das ist eine der ersten Aufgaben: Du zeichnest ein ER-Modell. Wir entscheiden danach gemeinsam, welche Beziehungen tatsächlich 1:1, 1:n oder M:n sind.

# 5. Was ist eine „Komponente“?

Mit Komponente ist hier ein technischer Baustein des Systems gemeint. Nicht jede Komponente ist eine einzelne Java-Klasse. Ein System kann aus mehreren größeren Komponenten bestehen.

| Komponente | Aufgabe |
|---|---|
| Browser / Frontend | Zeigt die Benutzeroberfläche und sendet Requests an das Backend. |
| Spring-Boot-Backend | Nimmt Requests an, prüft sie, führt Geschäftslogik aus und liefert Responses. |
| Datenbank | Speichert Benutzer, Workouts, Nachrichten usw. dauerhaft. |

| Komponente | Aufgabe |
|---|---|
| Authentication | Erkennt, wer eingeloggt ist. |
| Authorization | Prüft, ob der eingeloggte Benutzer eine Aktion ausführen darf. |
| Object Storage | Speichert hochgeladene Bilder. |
| Redis | Später Cache, Pub/Sub oder Grundlage für Hintergrundjobs. |
| Worker | Führt Aufgaben außerhalb des normalen HTTP-Requests aus, z. B. Bildverarbeitung. |
| Reverse Proxy | Nimmt Internetverkehr an und leitet ihn an die richtigen Dienste weiter; später auch TLS. |
| CI/CD | Baut, testet und deployt das Projekt automatisiert. |

# 6. Ein typischer Request – dein erster Entwurf korrigiert

Deine Aussage „Daten erfassen und per POST verschicken“ beschreibt nur einen Teil. Ein typischer Request kann beispielsweise so aussehen:

```
Browser → HTTPS → Server/Reverse Proxy → Spring-Boot-Controller → Authentifizierung → Validierung → Service/Business Logic → Repository/JPA → SQL → Datenbank → Response → Browser
```

Wichtig: POST ist eine HTTP-Methode, kein allgemeines „Internet-Protokoll“. HTTP ist das Protokoll. HTTPS bedeutet vereinfacht HTTP über eine TLS-gesicherte Verbindung.

Später untersuchen wir außerdem, was unter HTTP passiert: DNS, TCP/IP, Ports, TLS und Verbindungen. Dadurch wird dein vorhandenes Netzwerk-Wissen praktisch.

# 7. Ziel-Technologien

| Bereich | Technologie | Einsatz |
|---|---|---|
| Sprache | Java | Backend und Vertiefung deiner vorhandenen OOP-Kenntnisse. |
| Backend | Spring Boot | REST API, Dependency Injection, Security, Datenbankintegration. |
| Build | Maven | Abhängigkeiten, Build, Tests und Packaging. |
| DB | PostgreSQL | Primäre relationale Datenbank; dein vorhandenes MySQL-Wissen wird übertragen. |
| Persistence | JPA/Hibernate | Mapping zwischen Java-Objekten und relationalen Daten. |
| API | HTTP/REST + JSON | Kommunikation zwischen Frontend und Backend. |
| Frontend | HTML/CSS/JavaScript + React/TypeScript | Weboberfläche und Client-Logik. |
| Realtime | WebSocket | Live-Nachrichten/Benachrichtigungen. |
| Cache/Events | Redis | Später Cache, Pub/Sub und/oder Queue-Infrastruktur. |
| Dateien | S3-kompatibler Object Storage | Bilder und andere Uploads. |
| Tests | JUnit + Mockito + Integration Tests + Playwright | Unit-, Integrations- und End-to-End-Tests. |
| Ops | Linux + Docker + Reverse Proxy | Ausführen und Deployen des Systems. |
| CI/CD | GitHub Actions | Automatisiertes Testen und Bauen/Deployen. |
| Observability | Logging; später Metrics/Tracing | Fehleranalyse und Betrieb. |

# 8. Lern- und Implementierungsplan

## Phase 0 – Anforderungen und Architektur

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 0.1 | Projektproblem und Zielgruppe in 5–10 Sätzen beschreiben. | Requirements Engineering |
| Auftrag 0.2 | Benutzerrollen und Rechte als Tabelle definieren. | Authorization / Rollenmodell |
| Auftrag 0.3 | Erste Entitätenliste erstellen: User, Workout, Exercise, Set, Nachricht usw. | Domänenmodell |
| Auftrag 0.4 | ER-Diagramm zeichnen und 1:1, 1:n, M:n markieren. | Datenmodellierung |
| Auftrag 0.5 | Einen Request als Sequenzdiagramm skizzieren. | HTTP / Architektur |
| Meilenstein | Architektur-Review mit mir: Wir prüfen deine Skizzen, bevor du programmierst. | Engineering Design |

## Phase 1 – Java + Spring Boot Fundament

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 1.1 | Maven-Projekt erstellen und Spring Boot starten. | Buildsystem / Framework |
| Auftrag 1.2 | GET /api/health implementieren. | HTTP / REST |
| Auftrag 1.3 | Controller, Service und Repository als Verantwortlichkeiten kennenlernen. | Layered Architecture |
| Auftrag 1.4 | DTOs und Request-Validation einführen. | API Design |
| Meilenstein | Eine sauber strukturierte REST-Anwendung läuft lokal. | Backend |

## Phase 2 – Datenbank

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 2.1 | ER-Modell in Tabellen übersetzen. | Relationales Modell |
| Auftrag 2.2 | PostgreSQL lokal einrichten. | DBMS |
| Auftrag 2.3 | Migrations kennenlernen und Tabellen versionieren. | Database Engineering |
| Auftrag 2.4 | User und Workout persistieren. | JPA / SQL |
| Auftrag 2.5 | JOINs, Constraints, Indexes und Transactions an realen Fällen untersuchen. | SQL |
| Meilenstein | Daten können über die API dauerhaft gespeichert und gelesen werden. | Persistence |

## Phase 3 – Frontend

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 3.1 | HTML/CSS/JS-Grundlagen an einer kleinen Oberfläche anwenden. | Web Basics |
| Auftrag 3.2 | React + TypeScript aufsetzen. | Frontend |
| Auftrag 3.3 | Login- und Workout-Seiten bauen. | UI / API Client |
| Auftrag 3.4 | API-Requests vom Frontend an das Backend senden. | HTTP |
| Meilenstein | Ein Benutzer kann im Browser ein Workout anlegen und wieder ansehen. | Full Stack |

## Phase 4 – Authentication & Security

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 4.1 | Registrierung und Login entwerfen. | Authentication |
| Auftrag 4.2 | Passwörter sicher hashen; niemals Klartext speichern. | Security |
| Auftrag 4.3 | Session-/Token-Konzept umsetzen. | Authentication |
| Auftrag 4.4 | Rollen und Berechtigungen implementieren. | Authorization |
| Auftrag 4.5 | Input Validation, CORS, CSRF/XSS und SQL Injection anhand des Projekts verstehen. | Web Security |
| Meilenstein | Geschützte Ressourcen können nur von berechtigten Benutzern erreicht werden. | Security |

## Phase 5 – Kernfunktionen

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 5.1 | Trainingspläne erstellen/bearbeiten. | CRUD / Domain Logic |
| Auftrag 5.2 | Gewichte, Wiederholungen und Sets speichern. | Relationales Modell |
| Auftrag 5.3 | Körpergewicht als Zeitreihe speichern. | Datenmodell / Aggregation |
| Auftrag 5.4 | Aufgaben verwalten. | CRUD / State |
| Auftrag 5.5 | Nachrichten senden, bearbeiten, löschen und als gelesen markieren. | Domain Logic |
| Meilenstein | Die Kernanwendung ist funktional nutzbar. | MVP |

## Phase 6 – Dateien und Echtzeit

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 6.1 | Bilder hochladen und Metadaten speichern. | HTTP Multipart / Storage |
| Auftrag 6.2 | Object Storage statt Bild-BLOBs in der relationalen DB verwenden. | System Design |
| Auftrag 6.3 | WebSocket-Verbindung aufbauen. | Realtime Networking |
| Auftrag 6.4 | Live-Benachrichtigung bei neuer Nachricht implementieren. | WebSockets / Events |
| Meilenstein | Die Anwendung demonstriert HTTP/REST und bidirektionale Echtzeitkommunikation. | Networking |

## Phase 7 – Redis und Hintergrundjobs

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 7.1 | Redis als Cache kennenlernen. | Caching |
| Auftrag 7.2 | Ein sinnvolles Cache-Szenario implementieren. | Performance |
| Auftrag 7.3 | Hintergrundjob für Bildverarbeitung/Thumbnailing entwerfen. | Async Processing |
| Auftrag 7.4 | Retries und Fehlerfälle behandeln. | Reliability |
| Meilenstein | Nicht jede Arbeit muss innerhalb des HTTP-Requests passieren. | Distributed Components |

## Phase 8 – Testing und Qualität

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 8.1 | Unit Tests für reine Business-Logik schreiben. | JUnit |
| Auftrag 8.2 | Integration Test für API + Datenbank schreiben. | Integration Testing |
| Auftrag 8.3 | End-to-End-Workflow im Browser testen. | Playwright / E2E |
| Auftrag 8.4 | Fehlerfälle und Berechtigungsfehler testen. | Security / Quality |

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Meilenstein | Wichtige Funktionen sind automatisiert abgesichert. | Software Quality |

## Phase 9 – Linux, Docker und Deployment

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 9.1 | Linux-Prozesse, Ports, Logs und Environment Variables praktisch untersuchen. | Operating Systems |
| Auftrag 9.2 | Backend und Datenbank containerisieren. | Docker |
| Auftrag 9.3 | Reverse Proxy einrichten. | Networking / Deployment |
| Auftrag 9.4 | HTTPS/TLS und Domain verstehen und einrichten. | Security / Networking |
| Auftrag 9.5 | Anwendung auf einem Linux-Server deployen. | Operations |
| Meilenstein | Die Anwendung läuft außerhalb deines Entwicklungsrechners. | Deployment |

## Phase 10 – CI/CD und Observability

| Auftrag | Was du machst | Konzept |
|---|---|---|
| Auftrag 10.1 | GitHub Actions für Build, Tests und Type-/Style-Checks einrichten. | CI |
| Auftrag 10.2 | Deployment-Pipeline definieren. | CD |
| Auftrag 10.3 | Strukturiertes Logging mit Request-ID einführen. | Observability |
| Auftrag 10.4 | Fehler und Performance-Metriken sichtbar machen. | Monitoring |
| Meilenstein | Ein Push löst automatisierte Qualitätsprüfungen aus; das System ist betreibbar. | Professional Engineering |

# 9. Architektur-Zielbild

Das ist das Zielbild, nicht die Architektur des ersten Tages:

```
Browser / React
↓ HTTPS / HTTP
Reverse Proxy
↓
Spring Boot API
├── Authentication / Authorization
├── Controller
├── Service / Business Logic
├── Repository / JPA
├── WebSocket Endpoint
└── Background Jobs
↓
PostgreSQL + Redis + Object Storage
```

Wir bauen dieses System inkrementell. Jede neue Komponente bekommt erst dann einen Platz, wenn ein konkretes Problem oder eine Lernanforderung sie rechtfertigt.

# 10. Qualitätsregeln für das Projekt

• Kein Copy-Paste aus Tutorials als Endlösung. Tutorials dürfen zum Lernen dienen; der finale Code muss von dir verstanden werden.

• Jede größere Funktion beginnt mit einer kurzen technischen Skizze.

• Jede Architekturentscheidung bekommt eine Begründung.

• Git wird von Anfang an benutzt; sinnvolle Commits statt eines riesigen Abschluss-Commits.

• Fehler werden nicht einfach umgangen. Wir untersuchen Ursache, Wirkung und Fix.

• Sicherheit ist Bestandteil der Entwicklung und kein nachträgliches Extra.

• Tests werden zusammen mit wichtigen Funktionen aufgebaut, nicht erst ganz am Ende.

• Keine unnötige Technologie nur fürs Portfolio. Jede Komponente muss einen Zweck haben.

# 11. Deine erste konkrete Aufgabe

Noch nicht programmieren. Dein nächster Auftrag ist die Modellierung.

A. Zeichne die Entitäten, die du für Version 1 brauchst. Starte mit: User, Role, Workout, Exercise, WorkoutExercise, Set, Task, Message, BodyWeight, Image. Ergänze nur, wenn du einen guten Grund hast.

B. Verbinde die Entitäten mit 1:1, 1:n oder M:n.

C. Schreibe zu jeder Entität 3–10 wichtige Attribute.

D. Zeichne einen Ablauf für: „Benutzer loggt sich ein und speichert ein Workout mit zwei Übungen und mehreren Sätzen.“

E. Zeichne zusätzlich den Netzwerkweg grob: Browser → ? → ? → Datenbank. Schreibe daneben, welche Protokolle du an den einzelnen Stellen vermutest.

Schick mir diese Skizze anschließend hier – auch als einfache ASCII-Zeichnung oder Foto/Diagramm. Ich reviewe sie, stelle Rückfragen und wir gehen erst danach an die Implementierung.

# 12. Definition of Done für Version 1.0

• Ein Benutzer kann sich registrieren und anmelden.

• Benutzerrollen und Berechtigungen funktionieren.

- Trainingspläne und Trainingsdaten werden persistent gespeichert.
- Aufgaben und Nachrichten funktionieren.
- Bilder können sicher hochgeladen und wieder angezeigt werden.
- Das Frontend kommuniziert über eine dokumentierte API mit dem Backend.
- Wichtige Kernfunktionen besitzen automatisierte Tests.
- Die Anwendung läuft reproduzierbar über Docker.
- Die Anwendung kann auf einem Linux-System betrieben werden.
- README, Architekturdiagramm, Datenmodell, API-Dokumentation und technische Entscheidungen sind dokumentiert.

Hinweis: Dieser Plan ist bewusst anspruchsvoll. Nicht jede Phase muss in einem Zug abgeschlossen werden. Das Ziel ist nicht, möglichst schnell möglichst viele Technologien abzuhaken, sondern ein System zu bauen, bei dem du am Ende erklären kannst, warum die einzelnen Teile existieren und wie sie zusammenarbeiten.